

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 4

4 Programming in C and C++ (djc11)

strings

- (a) BCPL was a precursor to C. It stored character strings using a byte initialised to l , the length of the string, with the characters of the string held in the following bytes. What differences arise from the C method for storing character strings? [3 marks]

Answer: In C, the string length is defined by the first occurrence of the null character (code 0). The differences arising are that nulls can be stored in a BCPL string but their length is restricted to 255 characters (or fewer using UTF-8). The string length operator varies between constant and linear cost. In C, the suffix of a string is a valid string, but not in BCPL.

memory
management

- (b) A dictionary data structure in C stores key/value pairs where both elements are character strings. Entries are created or updated using

```
void insert(const char *key, const char *value)
```

Discuss whether the caller or callee should be responsible for allocating store for the strings. What consequences could arise if a caller mutates its local copy of a key? [5 marks]

Answer: Ownership could potentially be different between keys and values. If the caller is responsible for allocating store for the strings, it must not de-allocate the key memory until it is sure that the key/value store is no longer going to be used and the value memory until it is sure that key will not be further consulted, otherwise there will be dangling references. Moreover, it must not mutate the key strings since pretty much any indexing function used within the store, that is more advanced than linear search, is likely to have its ordering assumptions violated if keys are mutated.

To support caller allocation, we can argue the caller had it anyway for both key and value: this can save space. For the callee argument, we instead argue that none of the above restrictions apply and the additional space used is not typically any concern for keys, which tend to be small, but might be a worry for exceptionally large values.

A real-world C system might ensure all strings are only held once in a central string store, of which our key/value store is just one customer. This manages memory centrally and enables rapid determination of string identity just on pointer comparison.

linking

- (c) Students have been told that, in many situations, C array declarations are interchangeable with C pointer declarations. Rather than using a shared header file, a student writes `extern char *mydata;` at top-level directly in a `.c` file. Their program does not work. A friend changes it to `extern char mydata[];`. Might this change make any difference to compiler errors/warnings or run-time operation? Would a shared header file (`.h`) help? [6 marks]

Answer: While the equivalence exists in function parameter lists, it certainly is not true at top-level. Array declarations degenerate to pointers when passed as arguments to functions. Differences include the behaviour of the `sizeof()` operator and the mutability of the handle.

Note that the linker does not store signatures/types for C (but does for C++ owing to squirrelling.) We deduce the underlying definition is `char mydata[];`, and that the student's

original code had a misdeclaration that caused everything to compile cleanly but most likely gave a segmentation fault at run time (because the first four (or eight) bytes of the array would be interpreted as a pointer). The difference is the imported symbol changes from being the address of a statically-allocated array to the address of a pointer that has been initialised to that value. Both declarations will compile without error and link too, but only one matches the definition. A header file that contains the correct declaration which is imported both by the definer and the client will have helpfully caused a compile-time error/warning if the definition disagreed with the declaration.

casts

- (d) Stating any assumptions made, and considering endianness, describe what happens inside the compiler and at run time for each of the following type casts.

```
extern double f1;
int i1 = (int)f1;
char *p2 = (char *)&i1;
int i3 = (int)(*p2);
```

[6 marks]

Answer: *i)* There is no indication that `f1` is a compile-time constant, so it will likely be converted from double to integer form at run time using a floating-point to integer conversion instruction (or run-time subroutine). The result is stored in `i1`.

ii) The cast of an integer address to a char pointer will not generate a compile-time warning and, at run time, `p2` will be loaded with the address of `i1` (which is likely to be a stack location since this code fragment is presumably a function body). The `char` location pointed at will depend on the endianness of the system: either the most-significant or least-significant byte of `i1`

iii) The explicit cast of the r.h.s. of the assign to `i3` has no effect at compile time or run time since it is also implicit/unnecessary as this is a widening operation. The C standard does not specify whether an unqualified `char` is signed or unsigned; at run-time x86, implementations of C commonly sign-extend (copying the m.s.b. of the character into all of the new bit positions) the value of `*p`, while ARM implementations (apart from iOS) typically zero-extend. For example, if `f3` were 257.0, then `i1=257`, `*p2=1`, `i3=1`.
